Modèles algorithmiques fondamentaux sur structures à accès direct

Thème principal : Algorithmique et Programmation - Structures de données et modèles de parcours

Synthèse exécutive : Ce cours introduit les trois grands modèles algorithmiques indispensables à maîtriser en BUT 1 Informatique : le parcours complet avec traitement systématique, le parcours complet avec traitement conditionné, et la recherche de première occurrence. L'objectif est de fournir des schémas de solutions réutilisables et rigoureux, applicables en particulier sur des structures à accès direct (comme les tableaux et les chaînes de caractères), en insistant sur l'analyse préalable des structures, la gestion des bornes et la rigueur d'écriture pour réussir les SAE (Semaine d'Apprentissage et d'Évaluation).

1. Le Pourquoi & Les Mécanismes d'Accès aux Structures de Données

Avant de concevoir un algorithme, il est impératif d'identifier la nature de la collection de données que l'on manipule. Le choix d'un modèle algorithmique dépend directement des opérations permises par la structure. Si l'on travaille sur une structure à accès direct (tableaux, chaînes de caractères), on peut cibler n'importe quel indice immédiatement. En revanche, sur un fichier séquentiel ou une liste chaînée simple (accès séquentiel), il est impossible de sauter directement à la case 50 : il faut obligatoirement passer par les cases 0 à 49. Les modèles présentés dans ce cours reposent principalement sur les structures à accès direct, tandis que d'autres modules (assurés par M. Maxime Masson) se concentrent sur les accès séquentiels.

- Structure de données : Collection regroupant un ensemble d'éléments de même type (caractères, entiers, etc.).
- Accès direct : Mécanisme permettant d'accéder instantanément à n'importe quel élément via son indice ou sa clé, sans parcourir les précédents.
- Accès séquentiel : Mécanisme nécessitant de se positionner au début puis de progresser pas à pas d'un élément au suivant.
- Accès dynamique : Structure proposant à la fois des mécanismes d'accès séquentiel et d'accès direct.

Cas concret enseignant : *Le professeur insiste sur une erreur classique en partiel : appliquer un modèle nécessitant un accès direct (comme l'accès par indice dans un tableau) sur un fichier texte ou un fichier CSV brut qui ne propose qu'un flux séquentiel, ce qui invalide totalement l'algorithme.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle est le plus simple mais exige une rigueur absolue sur la gestion des indices. Il est utilisé lorsque l'on doit inspecter et modifier ou afficher l'intégralité d'une zone de données (par exemple, remplir toutes les cases d'un tableau, mettre en majuscules tous les caractères d'une chaîne). Dans un tableau de N éléments en langage algorithmique standard, les indices vont de 0 à N - 1. Le piège classique est de faire varier la boucle jusqu'à N, ce qui provoque une erreur fatale de dépassement de capacité (out of bounds).

- Objectif : Appliquer un traitement à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : Une boucle bornée (pour / 'for').
- Indice : Variable de boucle variant de la borne initiale à la borne finale.

Cas concret enseignant : *Exemple concret : Afficher un tableau de N cases. L'indice 'i' varie de 0 à N - 1. Le traitement répété consiste simplement à afficher l'élément du tableau situé à la position 'i'.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

C'est une évolution directe du modèle précédent. Le parcours reste complet (on visite tous les éléments de la borne A à la borne B), mais l'action à l'intérieur de la boucle est conditionnée par un test logique. Par exemple, compter le

nombre de voyelles dans une chaîne de caractères ou afficher uniquement les nombres positifs d'un tableau. La condition peut être simple (ex: supérieur à 0) ou complexe (cumulant plusieurs critères, voire impliquant les éléments voisins si l'algorithme est plus sophistiqué).

- Objectif : Parcourir l'ensemble d'une structure (de la borne de début à la fin) mais n'appliquer le traitement qu'aux éléments vérifiant une propriété spécifique.
- Structure : Boucle bornée (pour) englobant une structure alternative (si / 'if').

Cas concret enseignant : *Analogie du contrôle qualité : Sur une chaîne de montage (le parcours complet), on examine chaque pièce, mais on ne retire de la chaîne (traitement) que les pièces présentant un défaut précis.*

4. Troisième Modèle : La Recherche de Première Occurrence

Contrairement aux deux premiers modèles, il ne s'agit pas de parcourir forcément toute la structure, mais de s'arrêter dès que l'objectif est atteint. C'est un algorithme pessimiste par nature : on initialise un booléen 'trouvé' à faux, considérant que l'élément n'est pas là, et on ajuste notre certitude au fil de la visite. Il existe deux variantes principales : la variante 1 (boucle 'tant que' avec rupture immédiate dès la découverte, très utile si l'on cherche la position exacte de l'élément) et la variante 2 (regroupant les conditions d'arrêt en tête de boucle, plus simple à structurer syntaxiquement mais s'arrêtant moins intuitivement sur la position exacte).

- Objectif : Trouver le tout premier élément d'une structure qui vérifie une propriété donnée.
- Itération non bornée : Utilisation d'une boucle conditionnelle ('tant que' / 'while').
- Double condition d'arrêt : (1) A-t-on épuisé tous les éléments de la structure ? (2) A-t-on trouvé l'élément recherché ?
- Post-position : Permet souvent de s'arrêter précisément sur l'indice de l'élément trouvé (variante 1).

Cas concret enseignant : *Le professeur rappelle que rédiger un algorithme de recherche avec une seule condition d'arrêt est une erreur fondamentale en évaluation. Un algorithme de recherche doit obligatoirement savoir s'arrêter si la structure est entièrement parcourue en vain, ou s'il a trouvé le résultat avant la fin.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Une structure à accès dynamique propose à la fois des accès séquentiels et des accès direct. On peut donc tout à fait exploiter ses capacités d'accès direct pour appliquer un parcours systématique par indice.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que le choix dépend de ce que l'on cherche à faire. Si l'on a simplement besoin de savoir si un élément existe (vrai ou faux), la variante 2 avec un 'tant que' global est plus simple à coder. En revanche, si l'on a besoin de récupérer la position exacte (l'indice) du premier élément trouvé, la variante 1 est préférable car elle laisse l'itérateur positionné directement sur l'élément valide au moment de la rupture.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : C'est une adaptation un peu plus sophistiquée du même modèle de base. Le schéma général du parcours complet conditionné reste identique, mais le test logique à l'intérieur évalue l'élément courant 'i' en relation avec 'i-1' ou 'i+1', en veillant à ne pas provoquer de dépassement de bornes aux extrémités.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en utilisant le modèle approprié.

Résolution pas à pas :
1. Analyse du problème : On doit traiter tous les éléments d'un tableau (accès direct) de la première à la dernière case, mais le traitement (l'affichage) ne doit s'exécuter que si la valeur est strictement positive. Le modèle adapté est donc le 'Parcours complet avec traitement conditionnel'. 2. Détermination des bornes : Le tableau possède 'N' cases, indicées de 0 à N - 1. La borne de début est 0, la borne de fin est N - 1. 3. Construction de la boucle : Une boucle bornée 'pour i allant de 0 à N - 1'. 4. Ajout de la condition : À l'intérieur de la boucle, on ajoute un test : 'si T[i] > 0 alors'. 5. Action : 'Afficher(T[i])'.

Résultat : Algorithme final : Pour i allant de 0 à N - 1 faire Si T[i] > 0 alors Afficher(T[i]) Fin Si Fin Pour Ce code respecte strictement les prérequis d'un parcours complet conditionné sur structure à accès direct.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on utiliser pour vérifier si un tableau contient au moins une fois la valeur 10 ?

R : Une recherche de première occurrence, car on cherche à détecter la présence d'au moins un élément vérifiant une propriété et on peut s'arrêter dès qu'on le trouve.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce que les indices en informatique débutent à 0. Un tableau de N éléments possède des cases numérotées de 0 à N - 1. Utiliser N provoquerait une erreur d'indice hors limites.

Q : Combien de conditions d'arrêt minimales doit comporter un algorithme de recherche de première occurrence ?

R : Il doit obligatoirement comporter au minimum deux conditions d'arrêt : (1) A-t-on épuisé tous les éléments sans succès (recherche infructueuse) ? (2) A-t-on trouvé l'élément recherché (succès) ?